Sign in

pytorch / pytorch Public

Notifications Fork 23.6k Star 87.9k

Code Issues 5k+ Pull requests 1.1k Actions Projects 12

v2.6.0

pytorch / torch / nn / modules

Go to file

/ container.py

ringohoffman and pytorchmergebot nn.ModuleList.__getitem__ overloads (#132834) 7ea8374 · 7 months ago

976 lines (805 loc) · 34.2 KB .

Code Blame

Raw

```
2   # mypy: allow-untyped-defs
3   import operator
4   from collections import abc as container_abcs, OrderedDict
5   from itertools import chain, islice
6   from typing import (
7       Any,
8       Dict,
9       Iterable,
10      Iterator,
11      Mapping,
12      Optional,
13      overload,
```

```python
    Tuple,
    TypeVar,
    Union,
)
from typing_extensions import deprecated, Self

import torch
from torch._jit_internal import _copy_to_script_wrapper
from torch.nn.parameter import Parameter

from .module import Module


__all__ = [
    "Container",
    "Sequential",
    "ModuleList",
    "ModuleDict",
    "ParameterList",
    "ParameterDict",
]

T = TypeVar("T", bound=Module)


# Copied from torch.nn.modules.module, required for a custom __repr__ for ModuleList
def _addindent(s_, numSpaces):
    s = s_.split("\n")
    # don't do anything for single-line stuff
    if len(s) == 1:
        return s_
    first = s.pop(0)
    s = [(numSpaces * " ") + line for line in s]
    s = "\n".join(s)
    s = first + "\n" + s
    return s


```

```
@deprecated(
    "`nn.Container` is deprecated. "
    "All of it's functionality is now implemented in `nn.Module`. Subclass that instead.",
    category=FutureWarning,
)
class Container(Module):
    def __init__(self, **kwargs: Any) -> None:
        super().__init__()
        for key, value in kwargs.items():
            self.add_module(key, value)


class Sequential(Module):
    r"""A sequential container.

    Modules will be added to it in the order they are passed in the
    constructor. Alternatively, an ``OrderedDict`` of modules can be
    passed in. The ``forward()`` method of ``Sequential`` accepts any
    input and forwards it to the first module it contains. It then
    "chains" outputs to inputs sequentially for each subsequent module,
    finally returning the output of the last module.

    The value a ``Sequential`` provides over manually calling a sequence
    of modules is that it allows treating the whole container as a
    single module, such that performing a transformation on the
    ``Sequential`` applies to each of the modules it stores (which are
    each a registered submodule of the ``Sequential``).

    What's the difference between a ``Sequential`` and a
    :class:`torch.nn.ModuleList`? A ``ModuleList`` is exactly what it
    sounds like--a list for storing ``Module`` s! On the other hand,
    the layers in a ``Sequential`` are connected in a cascading way.

    Example::

        # Using Sequential to create a small model. When `model` is run,
        # input will first be passed to `Conv2d(1,20,5)`. The output of
        # `Conv2d(1,20,5)` will be used as the input to the first
        # `ReLU`; the output of the first `ReLU` will become the input
```

```
            # for `Conv2d(20,64,5)`. Finally, the output of
            # `Conv2d(20,64,5)` will be used as input to the second `ReLU`
            model = nn.Sequential(
                      nn.Conv2d(1,20,5),
                      nn.ReLU(),
                      nn.Conv2d(20,64,5),
                      nn.ReLU()
                    )

            # Using Sequential with OrderedDict. This is functionally the
            # same as the above code
            model = nn.Sequential(OrderedDict([
                      ('conv1', nn.Conv2d(1,20,5)),
                      ('relu1', nn.ReLU()),
                      ('conv2', nn.Conv2d(20,64,5)),
                      ('relu2', nn.ReLU())
                    ]))
        """

        _modules: Dict[str, Module]  # type: ignore[assignment]

        @overload
        def __init__(self, *args: Module) -> None:
            ...

        @overload
        def __init__(self, arg: "OrderedDict[str, Module]") -> None:
            ...

        def __init__(self, *args):
            super().__init__()
            if len(args) == 1 and isinstance(args[0], OrderedDict):
                for key, module in args[0].items():
                    self.add_module(key, module)
            else:
                for idx, module in enumerate(args):
                    self.add_module(str(idx), module)

```

```
129     def _get_item_by_idx(self, iterator, idx) -> T:  # type: ignore[misc, type-var]
130         """Get the idx-th item of the iterator."""
131         size = len(self)
132         idx = operator.index(idx)
133         if not -size <= idx < size:
134             raise IndexError(f"index {idx} is out of range")
135         idx %= size
136         return next(islice(iterator, idx, None))
137 
138     @_copy_to_script_wrapper
139     def __getitem__(self, idx: Union[slice, int]) -> Union["Sequential", T]:
140         if isinstance(idx, slice):
141             return self.__class__(OrderedDict(list(self._modules.items())[idx]))
142         else:
143             return self._get_item_by_idx(self._modules.values(), idx)
144 
145     def __setitem__(self, idx: int, module: Module) -> None:
146         key: str = self._get_item_by_idx(self._modules.keys(), idx)
147         return setattr(self, key, module)
148 
149     def __delitem__(self, idx: Union[slice, int]) -> None:
150         if isinstance(idx, slice):
151             for key in list(self._modules.keys())[idx]:
152                 delattr(self, key)
153         else:
154             key = self._get_item_by_idx(self._modules.keys(), idx)
155             delattr(self, key)
156         # To preserve numbering
157         str_indices = [str(i) for i in range(len(self._modules))]
158         self._modules = OrderedDict(list(zip(str_indices, self._modules.values())))
159 
160     @_copy_to_script_wrapper
161     def __len__(self) -> int:
162         return len(self._modules)
163 
164     def __add__(self, other) -> "Sequential":
165         if isinstance(other, Sequential):
166             ret = Sequential()
```

```
167                 for layer in self:
168                     ret.append(layer)
169                 for layer in other:
170                     ret.append(layer)
171                 return ret
172             else:
173                 raise ValueError(
174                     "add operator supports only objects "
175                     f"of Sequential class, but {str(type(other))} is given."
176                 )
177 
178         def pop(self, key: Union[int, slice]) -> Module:
179             v = self[key]
180             del self[key]
181             return v
182 
183         def __iadd__(self, other) -> Self:
184             if isinstance(other, Sequential):
185                 offset = len(self)
186                 for i, module in enumerate(other):
187                     self.add_module(str(i + offset), module)
188                 return self
189             else:
190                 raise ValueError(
191                     "add operator supports only objects "
192                     f"of Sequential class, but {str(type(other))} is given."
193                 )
194 
195         def __mul__(self, other: int) -> "Sequential":
196             if not isinstance(other, int):
197                 raise TypeError(
198                     f"unsupported operand type(s) for *: {type(self)} and {type(other)}"
199                 )
200             elif other <= 0:
201                 raise ValueError(
202                     f"Non-positive multiplication factor {other} for {type(self)}"
203                 )
204             else:
205                 combined = Sequential()
```

```
            offset = 0
            for _ in range(other):
                for module in self:
                    combined.add_module(str(offset), module)
                    offset += 1
            return combined

    def __rmul__(self, other: int) -> "Sequential":
        return self.__mul__(other)

    def __imul__(self, other: int) -> Self:
        if not isinstance(other, int):
            raise TypeError(
                f"unsupported operand type(s) for *: {type(self)} and {type(other)}"
            )
        elif other <= 0:
            raise ValueError(
                f"Non-positive multiplication factor {other} for {type(self)}"
            )
        else:
            len_original = len(self)
            offset = len(self)
            for _ in range(other - 1):
                for i in range(len_original):
                    self.add_module(str(i + offset), self._modules[str(i)])
                offset += len_original
            return self

    @_copy_to_script_wrapper
    def __dir__(self):
        keys = super().__dir__()
        keys = [key for key in keys if not key.isdigit()]
        return keys

    @_copy_to_script_wrapper
    def __iter__(self) -> Iterator[Module]:
        return iter(self._modules.values())

```

```
244         # NB: We can't really type check this function as the type of input
245         # may change dynamically (as is tested in
246         # TestScript.test_sequential_intermediary_types).  Cannot annotate
247         # with Any as TorchScript expects a more precise type
248         def forward(self, input):
249             for module in self:
250                 input = module(input)
251             return input
252 
253         def append(self, module: Module) -> "Sequential":
254             r"""Append a given module to the end.
255 
256             Args:
257                 module (nn.Module): module to append
258             """
259             self.add_module(str(len(self)), module)
260             return self
261 
262         def insert(self, index: int, module: Module) -> "Sequential":
263             if not isinstance(module, Module):
264                 raise AssertionError(f"module should be of type: {Module}")
265             n = len(self._modules)
266             if not (-n <= index <= n):
267                 raise IndexError(f"Index out of range: {index}")
268             if index < 0:
269                 index += n
270             for i in range(n, index, -1):
271                 self._modules[str(i)] = self._modules[str(i - 1)]
272             self._modules[str(index)] = module
273             return self
274 
275         def extend(self, sequential) -> "Sequential":
276             for layer in sequential:
277                 self.append(layer)
278             return self
279 
280 
281     class ModuleList(Module):
```

```
r   Holds submodules in a list.

:class:`~torch.nn.ModuleList` can be indexed like a regular Python list, but
modules it contains are properly registered, and will be visible by all
:class:`~torch.nn.Module` methods.

Args:
    modules (iterable, optional): an iterable of modules to add

Example::

    class MyModule(nn.Module):
        def __init__(self) -> None:
            super().__init__()
            self.linears = nn.ModuleList([nn.Linear(10, 10) for i in range(10)])

        def forward(self, x):
            # ModuleList can act as an iterable, or be indexed using ints
            for i, l in enumerate(self.linears):
                x = self.linears[i // 2](x) + l(x)
            return x
"""

_modules: Dict[str, Module]  # type: ignore[assignment]

def __init__(self, modules: Optional[Iterable[Module]] = None) -> None:
    super().__init__()
    if modules is not None:
        self += modules

def _get_abs_string_index(self, idx):
    """Get the absolute index for the list of modules."""
    idx = operator.index(idx)
    if not (-len(self) <= idx < len(self)):
        raise IndexError(f"index {idx} is out of range")
    if idx < 0:
        idx += len(self)
    return str(idx)
```

```python
    @overload
    def __getitem__(self, idx: slice) -> "ModuleList":
        ...

    @overload
    def __getitem__(self, idx: int) -> Module:
        ...

    @_copy_to_script_wrapper
    def __getitem__(self, idx: Union[int, slice]) -> Union[Module, "ModuleList"]:
        if isinstance(idx, slice):
            return self.__class__(list(self._modules.values())[idx])
        else:
            return self._modules[self._get_abs_string_index(idx)]

    def __setitem__(self, idx: int, module: Module) -> None:
        idx = self._get_abs_string_index(idx)
        return setattr(self, str(idx), module)

    def __delitem__(self, idx: Union[int, slice]) -> None:
        if isinstance(idx, slice):
            for k in range(len(self._modules))[idx]:
                delattr(self, str(k))
        else:
            delattr(self, self._get_abs_string_index(idx))
        # To preserve numbering, self._modules is being reconstructed with modules after deletion
        str_indices = [str(i) for i in range(len(self._modules))]
        self._modules = OrderedDict(list(zip(str_indices, self._modules.values())))

    @_copy_to_script_wrapper
    def __len__(self) -> int:
        return len(self._modules)

    @_copy_to_script_wrapper
    def __iter__(self) -> Iterator[Module]:
        return iter(self._modules.values())

    def __iadd__(self, modules: Iterable[Module]) -> Self:
```

```
359             return self.extend(modules)
360
361         def __add__(self, other: Iterable[Module]) -> "ModuleList":
362             combined = ModuleList()
363             for i, module in enumerate(chain(self, other)):
364                 combined.add_module(str(i), module)
365             return combined
366
367         def __repr__(self):
368             """Return a custom repr for ModuleList that compresses repeated module representations."""
369             list_of_reprs = [repr(item) for item in self]
370             if len(list_of_reprs) == 0:
371                 return self._get_name() + "()"
372
373             start_end_indices = [[0, 0]]
374             repeated_blocks = [list_of_reprs[0]]
375             for i, r in enumerate(list_of_reprs[1:], 1):
376                 if r == repeated_blocks[-1]:
377                     start_end_indices[-1][1] += 1
378                     continue
379
380                 start_end_indices.append([i, i])
381                 repeated_blocks.append(r)
382
383             lines = []
384             main_str = self._get_name() + "("
385             for (start_id, end_id), b in zip(start_end_indices, repeated_blocks):
386                 local_repr = f"({start_id}): {b}"  # default repr
387
388                 if start_id != end_id:
389                     n = end_id - start_id + 1
390                     local_repr = f"({start_id}-{end_id}): {n} x {b}"
391
392                 local_repr = _addindent(local_repr, 2)
393                 lines.append(local_repr)
394
395             main_str += "\n  " + "\n  ".join(lines) + "\n"
396             main_str += ")"
397             return main_str
```

```
397             return main_str
398
399         @_copy_to_script_wrapper
400         def __dir__(self):
401             keys = super().__dir__()
402             keys = [key for key in keys if not key.isdigit()]
403             return keys
404
405         def insert(self, index: int, module: Module) -> None:
406             r"""Insert a given module before a given index in the list.
407
408             Args:
409                 index (int): index to insert.
410                 module (nn.Module): module to insert
411             """
412             for i in range(len(self._modules), index, -1):
413                 self._modules[str(i)] = self._modules[str(i - 1)]
414             self._modules[str(index)] = module
415
416         def append(self, module: Module) -> "ModuleList":
417             r"""Append a given module to the end of the list.
418
419             Args:
420                 module (nn.Module): module to append
421             """
422             self.add_module(str(len(self)), module)
423             return self
424
425         def pop(self, key: Union[int, slice]) -> Module:
426             v = self[key]
427             del self[key]
428             return v
429
430         def extend(self, modules: Iterable[Module]) -> Self:
431             r"""Append modules from a Python iterable to the end of the list.
432
433             Args:
434                 modules (iterable): iterable of modules to append
435             """
```

```
436             if not isinstance(modules, container_abcs.Iterable):
437                 raise TypeError(
438                     "ModuleList.extend should be called with an "
439                     "iterable, but got " + type(modules).__name__
440                 )
441             offset = len(self)
442             for i, module in enumerate(modules):
443                 self.add_module(str(offset + i), module)
444             return self
445 
446         # remove forward alltogether to fallback on Module's _forward_unimplemented
447 
448 
449     class ModuleDict(Module):
450         r"""Holds submodules in a dictionary.
451 
452         :class:`~torch.nn.ModuleDict` can be indexed like a regular Python dictionary,
453         but modules it contains are properly registered, and will be visible by all
454         :class:`~torch.nn.Module` methods.
455 
456         :class:`~torch.nn.ModuleDict` is an **ordered** dictionary that respects
457 
458         * the order of insertion, and
459 
460         * in :meth:`~torch.nn.ModuleDict.update`, the order of the merged
461           ``OrderedDict``, ``dict`` (started from Python 3.6) or another
462           :class:`~torch.nn.ModuleDict` (the argument to
463           :meth:`~torch.nn.ModuleDict.update`).
464 
465         Note that :meth:`~torch.nn.ModuleDict.update` with other unordered mapping
466         types (e.g., Python's plain ``dict`` before Python version 3.6) does not
467         preserve the order of the merged mapping.
468 
469         Args:
470             modules (iterable, optional): a mapping (dictionary) of (string: module)
471                 or an iterable of key-value pairs of type (string, module)
472 
473         Example::
```

```
474
475             class MyModule(nn.Module):
476                 def __init__(self) -> None:
477                     super().__init__()
478                     self.choices = nn.ModuleDict({
479                             'conv': nn.Conv2d(10, 10, 3),
480                             'pool': nn.MaxPool2d(3)
481                     })
482                     self.activations = nn.ModuleDict([
483                             ['lrelu', nn.LeakyReLU()],
484                             ['prelu', nn.PReLU()]
485                     ])
486
487                 def forward(self, x, choice, act):
488                     x = self.choices[choice](x)
489                     x = self.activations[act](x)
490                     return x
491         """
492
493         _modules: Dict[str, Module]  # type: ignore[assignment]
494
495         def __init__(self, modules: Optional[Mapping[str, Module]] = None) -> None:
496             super().__init__()
497             if modules is not None:
498                 self.update(modules)
499
500         @_copy_to_script_wrapper
501         def __getitem__(self, key: str) -> Module:
502             return self._modules[key]
503
504         def __setitem__(self, key: str, module: Module) -> None:
505             self.add_module(key, module)
506
507         def __delitem__(self, key: str) -> None:
508             del self._modules[key]
509
510         @_copy_to_script_wrapper
511         def __len__(self) -> int:
512             return len(self._modules)
```

```
513
514         @_copy_to_script_wrapper
515         def __iter__(self) -> Iterator[str]:
516             return iter(self._modules)
517
518         @_copy_to_script_wrapper
519         def __contains__(self, key: str) -> bool:
520             return key in self._modules
521
522         def clear(self) -> None:
523             """Remove all items from the ModuleDict."""
524             self._modules.clear()
525
526         def pop(self, key: str) -> Module:
527             r"""Remove key from the ModuleDict and return its module.
528
529             Args:
530                 key (str): key to pop from the ModuleDict
531             """
532             v = self[key]
533             del self[key]
534             return v
535
536         @_copy_to_script_wrapper
537         def keys(self) -> Iterable[str]:
538             r"""Return an iterable of the ModuleDict keys."""
539             return self._modules.keys()
540
541         @_copy_to_script_wrapper
542         def items(self) -> Iterable[Tuple[str, Module]]:
543             r"""Return an iterable of the ModuleDict key/value pairs."""
544             return self._modules.items()
545
546         @_copy_to_script_wrapper
547         def values(self) -> Iterable[Module]:
548             r"""Return an iterable of the ModuleDict values."""
549             return self._modules.values()
550
```

```
    def update(self, modules: Mapping[str, Module]) -> None:
        r"""Update the :class:`~torch.nn.ModuleDict` with key-value pairs from a mapping, overwriting existing keys.

        .. note::
            If :attr:`modules` is an ``OrderedDict``, a :class:`~torch.nn.ModuleDict`, or
            an iterable of key-value pairs, the order of new elements in it is preserved.

        Args:
            modules (iterable): a mapping (dictionary) from string to :class:`~torch.nn.Module`,
                or an iterable of key-value pairs of type (string, :class:`~torch.nn.Module`)
        """
        if not isinstance(modules, container_abcs.Iterable):
            raise TypeError(
                "ModuleDict.update should be called with an "
                "iterable of key/value pairs, but got " + type(modules).__name__
            )

        if isinstance(modules, (OrderedDict, ModuleDict, container_abcs.Mapping)):
            for key, module in modules.items():
                self[key] = module
        else:
            # modules here can be a list with two items
            for j, m in enumerate(modules):
                if not isinstance(m, container_abcs.Iterable):
                    raise TypeError(
                        "ModuleDict update sequence element "
                        "#" + str(j) + " should be Iterable; is" + type(m).__name__
                    )
                if not len(m) == 2:
                    raise ValueError(
                        "ModuleDict update sequence element "
                        "#" + str(j) + " has length " + str(len(m)) + "; 2 is required"
                    )
                # modules can be Mapping (what it's typed at), or a list: [(name1, module1), (name2, module2)]
                # that's too cumbersome to type correctly with overloads, so we add an ignore here
                self[m[0]] = m[1]  # type: ignore[assignment]

    # remove forward alltogether to fallback on Module's _forward_unimplemented
```

```
589
590
591 class ParameterList(Module):
592     r"""Holds parameters in a list.
593
594     :class:`~torch.nn.ParameterList` can be used like a regular Python
595     list, but Tensors that are :class:`~torch.nn.Parameter` are properly registered,
596     and will be visible by all :class:`~torch.nn.Module` methods.
597
598     Note that the constructor, assigning an element of the list, the
599     :meth:`~torch.nn.ParameterList.append` method and the :meth:`~torch.nn.ParameterList.extend`
600     method will convert any :class:`~torch.Tensor` into :class:`~torch.nn.Parameter`.
601
602     Args:
603         parameters (iterable, optional): an iterable of elements to add to the list.
604
605     Example::
606
607         class MyModule(nn.Module):
608             def __init__(self) -> None:
609                 super().__init__()
610                 self.params = nn.ParameterList([nn.Parameter(torch.randn(10, 10)) for i in range(10)])
611
612             def forward(self, x):
613                 # ParameterList can act as an iterable, or be indexed using ints
614                 for i, p in enumerate(self.params):
615                     x = self.params[i // 2].mm(x) + p.mm(x)
616                 return x
617     """
618
619     def __init__(self, values: Optional[Iterable[Any]] = None) -> None:
620         super().__init__()
621         self._size = 0
622         if values is not None:
623             self += values
624
625     def _get_abs_string_index(self, idx):
626         """Get the absolute index for the list of modules."""
627         idx = operator.index(idx)
```

```
628             if not (-len(self) <= idx < len(self)):
629                 raise IndexError(f"index {idx} is out of range")
630             if idx < 0:
631                 idx += len(self)
632             return str(idx)
633 
634         @overload
635         def __getitem__(self, idx: int) -> Any:
636             ...
637 
638         @overload
639         def __getitem__(self: T, idx: slice) -> T:
640             ...
641 
642         def __getitem__(self, idx):
643             if isinstance(idx, slice):
644                 start, stop, step = idx.indices(len(self))
645                 out = self.__class__()
646                 for i in range(start, stop, step):
647                     out.append(self[i])
648                 return out
649             else:
650                 idx = self._get_abs_string_index(idx)
651                 return getattr(self, str(idx))
652 
653         def __setitem__(self, idx: int, param: Any) -> None:
654             # Note that all other function that add an entry to the list part of
655             # the ParameterList end up here. So this is the only place where we need
656             # to wrap things into Parameter if needed.
657             # Objects added via setattr() are not in the list part and thus won't
658             # call into this function.
659             idx = self._get_abs_string_index(idx)
660             if isinstance(param, torch.Tensor) and not isinstance(param, Parameter):
661                 param = Parameter(param)
662             return setattr(self, str(idx), param)
663 
664         def __len__(self) -> int:
665             return self._size
```

```
666
667         def __iter__(self) -> Iterator[Any]:
668             return iter(self[i] for i in range(len(self)))
669
670         def __iadd__(self, parameters: Iterable[Any]) -> Self:
671             return self.extend(parameters)
672
673         def __dir__(self):
674             keys = super().__dir__()
675             keys = [key for key in keys if not key.isdigit()]
676             return keys
677
678         def append(self, value: Any) -> "ParameterList":
679             """Append a given value at the end of the list.
680
681             Args:
682                 value (Any): value to append
683             """
684             new_idx = len(self)
685             self._size += 1
686             self[new_idx] = value
687             return self
688
689         def extend(self, values: Iterable[Any]) -> Self:
690             """Append values from a Python iterable to the end of the list.
691
692             Args:
693                 values (iterable): iterable of values to append
694             """
695             # Tensor is an iterable but we never want to unpack it here
696             if not isinstance(values, container_abcs.Iterable) or isinstance(
697                 values, torch.Tensor
698             ):
699                 raise TypeError(
700                     "ParameterList.extend should be called with an "
701                     "iterable, but got " + type(values).__name__
702                 )
703             for value in values:
```

```
            self.append(value)
        return self

    def extra_repr(self) -> str:
        child_lines = []
        for k, p in enumerate(self):
            if isinstance(p, torch.Tensor):
                size_str = "x".join(str(size) for size in p.size())
                if p.device.type in ["cuda", torch._C._get_privateuse1_backend_name()]:
                    device_str = f" ({p.device})"
                else:
                    device_str = ""
                parastr = "{} containing: [{} of size {}{}]".format(
                    "Parameter" if isinstance(p, Parameter) else "Tensor",
                    p.dtype,
                    size_str,
                    device_str,
                )
                child_lines.append("  (" + str(k) + "): " + parastr)
            else:
                child_lines.append(
                    "  (" + str(k) + "): Object of type: " + type(p).__name__
                )

        tmpstr = "\n".join(child_lines)
        return tmpstr

    def __call__(self, *args, **kwargs):
        raise RuntimeError("ParameterList should not be called.")


class ParameterDict(Module):
    r"""Holds parameters in a dictionary.

    ParameterDict can be indexed like a regular Python dictionary, but Parameters it
    contains are properly registered, and will be visible by all Module methods.
    Other objects are treated as would be done by a regular Python dictionary

    :class:`~torch.nn.ParameterDict` is an **ordered** dictionary.
```

```
        :meth:`~torch.nn.ParameterDict.update` with other unordered mapping
        types (e.g., Python's plain ``dict``) does not preserve the order of the
        merged mapping. On the other hand, ``OrderedDict`` or another :class:`~torch.nn.ParameterDict`
        will preserve their ordering.

        Note that the constructor, assigning an element of the dictionary and the
        :meth:`~torch.nn.ParameterDict.update` method will convert any :class:`~torch.Tensor` into
        :class:`~torch.nn.Parameter`.

        Args:
            values (iterable, optional): a mapping (dictionary) of
                (string : Any) or an iterable of key-value pairs
                of type (string, Any)

        Example::

            class MyModule(nn.Module):
                def __init__(self) -> None:
                    super().__init__()
                    self.params = nn.ParameterDict({
                            'left': nn.Parameter(torch.randn(5, 10)),
                            'right': nn.Parameter(torch.randn(5, 10))
                    })

                def forward(self, x, choice):
                    x = self.params[choice].mm(x)
                    return x
        """

        def __init__(self, parameters: Any = None) -> None:
            super().__init__()
            self._keys: Dict[str, None] = {}
            if parameters is not None:
                self.update(parameters)

        def _key_to_attr(self, key: str) -> str:
            if not isinstance(key, str):
                raise TypeError(
```

```
781                 "Index given to ParameterDict cannot be used as a key as it is "
782                 f"not a string (type is '{type(key).__name__}'). Open an issue on "
783                 "github if you need non-string keys."
784             )
785         else:
786             # Use the key as-is so that `.named_parameters()` returns the right thing
787             return key
788
789     def __getitem__(self, key: str) -> Any:
790         attr = self._key_to_attr(key)
791         return getattr(self, attr)
792
793     def __setitem__(self, key: str, value: Any) -> None:
794         # Note that all other function that add an entry to the dictionary part of
795         # the ParameterDict end up here. So this is the only place where we need
796         # to wrap things into Parameter if needed.
797         # Objects added via setattr() are not in the dictionary part and thus won't
798         # call into this function.
799         self._keys[key] = None
800         attr = self._key_to_attr(key)
801         if isinstance(value, torch.Tensor) and not isinstance(value, Parameter):
802             value = Parameter(value)
803         setattr(self, attr, value)
804
805     def __delitem__(self, key: str) -> None:
806         del self._keys[key]
807         attr = self._key_to_attr(key)
808         delattr(self, attr)
809
810     def __len__(self) -> int:
811         return len(self._keys)
812
813     def __iter__(self) -> Iterator[str]:
814         return iter(self._keys)
815
816     def __reversed__(self) -> Iterator[str]:
817         return reversed(list(self._keys))
818
```

```python
    def copy(self) -> "ParameterDict":
        """Return a copy of this :class:`~torch.nn.ParameterDict` instance."""
        # We have to use an OrderedDict because the ParameterDict constructor
        # behaves differently on plain dict vs OrderedDict
        return ParameterDict(OrderedDict((k, self[k]) for k in self._keys))

    def __contains__(self, key: str) -> bool:
        return key in self._keys

    def setdefault(self, key: str, default: Optional[Any] = None) -> Any:
        """Set the default for a key in the Parameterdict.

        If key is in the ParameterDict, return its value.
        If not, insert `key` with a parameter `default` and return `default`.
        `default` defaults to `None`.

        Args:
            key (str): key to set default for
            default (Any): the parameter set to the key
        """
        if key not in self:
            self[key] = default
        return self[key]

    def clear(self) -> None:
        """Remove all items from the ParameterDict."""
        for k in self._keys.copy():
            del self[k]

    def pop(self, key: str) -> Any:
        r"""Remove key from the ParameterDict and return its parameter.

        Args:
            key (str): key to pop from the ParameterDict
        """
        v = self[key]
        del self[key]
        return v

```

```
858     def popitem(self) -> Tuple[str, Any]:
859         """Remove and return the last inserted `(key, parameter)` pair from the ParameterDict."""
860         k, _ = self._keys.popitem()
861         # We need the key in the _keys to be able to access/del
862         self._keys[k] = None
863         val = self[k]
864         del self[k]
865         return k, val
866
867     def get(self, key: str, default: Optional[Any] = None) -> Any:
868         r"""Return the parameter associated with key if present. Otherwise return default if provided, None if not.
869
870         Args:
871             key (str): key to get from the ParameterDict
872             default (Parameter, optional): value to return if key not present
873         """
874         return self[key] if key in self else default
875
876     def fromkeys(
877         self, keys: Iterable[str], default: Optional[Any] = None
878     ) -> "ParameterDict":
879         r"""Return a new ParameterDict with the keys provided.
880
881         Args:
882             keys (iterable, string): keys to make the new ParameterDict from
883             default (Parameter, optional): value to set for all keys
884         """
885         return ParameterDict((k, default) for k in keys)
886
887     def keys(self) -> Iterable[str]:
888         r"""Return an iterable of the ParameterDict keys."""
889         return self._keys.keys()
890
891     def items(self) -> Iterable[Tuple[str, Any]]:
892         r"""Return an iterable of the ParameterDict key/value pairs."""
893         return ((k, self[k]) for k in self._keys)
894
895     def values(self) -> Iterable[Any]:
```

```
896             r"""Return an iterable of the ParameterDict values."""
897             return (self[k] for k in self._keys)
898 
899         def update(self, parameters: Union[Mapping[str, Any], "ParameterDict"]) -> None:
900             r"""Update the :class:`~torch.nn.ParameterDict` with key-value pairs from ``parameters``, overwriting existing keys.
901 
902             .. note::
903                 If :attr:`parameters` is an ``OrderedDict``, a :class:`~torch.nn.ParameterDict`, or
904                 an iterable of key-value pairs, the order of new elements in it is preserved.
905 
906             Args:
907                 parameters (iterable): a mapping (dictionary) from string to
908                     :class:`~torch.nn.Parameter`, or an iterable of
909                     key-value pairs of type (string, :class:`~torch.nn.Parameter`)
910             """
911             if not isinstance(parameters, container_abcs.Iterable):
912                 raise TypeError(
913                     "ParametersDict.update should be called with an "
914                     "iterable of key/value pairs, but got " + type(parameters).__name__
915                 )
916 
917             if isinstance(parameters, (OrderedDict, ParameterDict)):
918                 for key, parameter in parameters.items():
919                     self[key] = parameter
920             elif isinstance(parameters, container_abcs.Mapping):
921                 for key, parameter in sorted(parameters.items()):
922                     self[key] = parameter
923             else:
924                 for j, p in enumerate(parameters):
925                     if not isinstance(p, container_abcs.Iterable):
926                         raise TypeError(
927                             "ParameterDict update sequence element "
928                             "#" + str(j) + " should be Iterable; is" + type(p).__name__
929                         )
930                     if not len(p) == 2:
931                         raise ValueError(
932                             "ParameterDict update sequence element "
933                             "#" + str(j) + " has length " + str(len(p)) + "; 2 is required"
```

```
                    )
                # parameters as length-2 list too cumbersome to type, see ModuleDict.update comment
                self[p[0]] = p[1]  # type: ignore[assignment]

    def extra_repr(self) -> str:
        child_lines = []
        for k, p in self.items():
            if isinstance(p, torch.Tensor):
                size_str = "x".join(str(size) for size in p.size())
                if p.device.type in ["cuda", torch._C._get_privateuse1_backend_name()]:
                    device_str = f" ({p.device})"
                else:
                    device_str = ""
                parastr = "{} containing: [{} of size {}{}]".format(
                    "Parameter" if isinstance(p, Parameter) else "Tensor",
                    torch.typename(p),
                    size_str,
                    device_str,
                )
                child_lines.append("  (" + str(k) + "): " + parastr)
            else:
                child_lines.append(
                    "  (" + str(k) + "): Object of type: " + type(p).__name__
                )
        tmpstr = "\n".join(child_lines)
        return tmpstr

    def __call__(self, input):
        raise RuntimeError("ParameterDict should not be called.")

    def __or__(self, other: "ParameterDict") -> "ParameterDict":
        copy = self.copy()
        copy.update(other)
        return copy

    def __ror__(self, other: "ParameterDict") -> "ParameterDict":
        copy = other.copy()
        copy.update(self)
        return copy
```

```
973
974         def __ior__(self, other: "ParameterDict") -> Self:
975             self.update(other)
976             return self
```